

PYTHON

Présenté par :
Xavier TABUTEAU

Les générateurs

Les générateurs

- Ce sont des itérables (comme les collections) mais aussi des itérateurs, ce qui implique qu'ils se consomment quand on les parcourt et que nous pouvons donc les parcourir qu'une fois (comme map ou filter). Ils sont généralement créés par des fonctions construites à l'aide du mot clef « yield ». Par abus de langage ces fonctions génératrices sont elles-mêmes appelées générateurs.
- Pour créer une fonction génératrice, elle doit contenir un ou plusieurs mot clé yield. Lors de l'appel, la fonction retournera un générateur, et appel à la fonction builtin « next() » sur la fonction génératrice, le code jusqu'au prochain yield sera exécuté. Puis la fonction reprendra à partir du yield lorsqu'un nouvel appel à ce générateur sera effectué. Comme pour tout itérateur, la fonction next appelle la méthode spéciale `__next__()` du générateur.
- Yield peut-être ou non suivi d'une expression. La valeur retournée par next sera celle du yield, ou None si aucune valeur ou expression n'est spécifiée après yield.

Les générateurs

Les générateurs

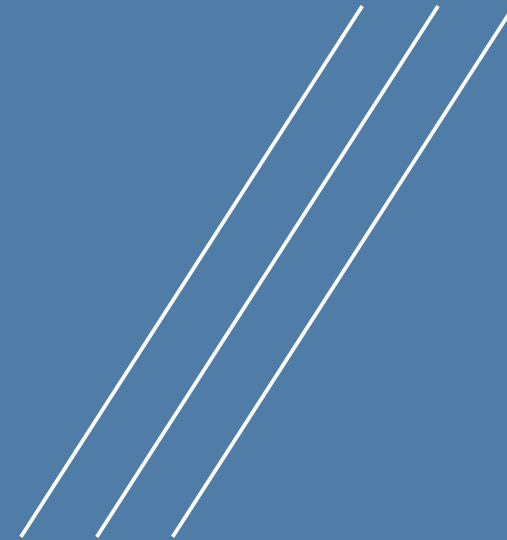
- Après la création d'un générateur, il est possible de le modifier via la méthode `send(paramètre)`. L'utilisation de `send` provoque la reprise de l'exécution de la fonction génératrice à partir du `yield` ou elle s'était arrêtée. Ce `yield` retourne à la fonction, le paramètre transmis via le `send`.
- Il est possible d'utiliser un sous-générateur (un générateur dans un autre) via le mot clé `yield from` « `sous_generateur()` ».
- Il est également possible créer des générateurs en compréhension, comme les listes, en remplaçant les `[]` par des `()`.
- La méthode `throw()` permet de lever une exception.
- La méthode `close()` d'un générateur permet de stopper celui ci.

Remarque : liste ou générateur ?

- Les listes prennent plus de place en mémoire car tous les éléments existent en même temps alors que le générateur n'a que le dernier élément généré en mémoire.
- Les générateurs ont une durée de vie plus courte car une fois itérés, ils disparaissent. Par conséquent on peut créer des générateurs avec des boucles infinies.
- Les générateurs ne sont pas indexables.

generateurs.py
ex_générateur

PYTHON



Présenté par
Xavier TABUTEAU